

CURSO DE ENGENHARIA DA COMPUTAÇÃO

SISTEMAS OPERACIONAIS

2026/2 — 8.º semestre de Engenharia da Computação

SIMULADOR DE ESCALONAMENTO DE TAREFAS

Documentação Técnica

Aline Cristina R. de Barros — RA 081230021

Gustavo A. Zaccheu — RA 081230023

Luis Gustavo de O. Caneiro — RA 081230029

Vitor Barbosa Carlos — RA 081230037

Projeto prático apresentado à disciplina de Sistemas Operacionais do curso de Engenharia da Computação, sob orientação do Prof. Vinicius S. Borges.

Professor: Vinicius S. Borges

Setembro de 2026

SUMÁRIO

1 VISÃO GERAL.....	4
2 SEPARAÇÃO ENTRE POLÍTICA E MECANISMO	4
3 DIAGRAMA DE MÓDULOS	5
4 ESTRUTURA DE UMA TAREFA	7
5 INTERFACE DOS MÓDULOS	8
5.1 Processo.__init__(id, chegada, duracao, prioridade, sc_inicio, sc_duracao)	8
5.2 Processo.adicionar_processamento(inicio, fim) → duracao_real.....	8
5.3 Processo.adicionar_troca_contexto(inicio, fim) → duracao.....	8
5.4 Processo.precisa_recurso() → bool	8
5.5 Processo.get_turnaround() → int.....	9
5.6 Processo.get_espera() → int	9
5.7 Processo.get_tempo_primeira_execucao() → int.....	9
5.8 gerar_cenario(quantidade, chegada_max, duracao_max, prioridade_max) → list[Processo].....	9
5.9 simular_escalonamento(processos, algoritmo, quantum_entry, ctx_time, alpha) → (media_execucao, media_espera, media_primeira_execucao, nome_processo).....	9
6 PARÂMETROS CONFIGURÁVEIS	9
7 FUNCIONAMENTO INTERNO.....	10
7.1 Laço de simulação	10
7.2 Métricas	11
8 CONVENÇÕES DE SIMULAÇÃO	12
9 DEPENDÊNCIAS E AMBIENTE	12
9.1 Executável.....	12
9.2 Execução via código-fonte.....	13

VISÃO GERAL

O simulador reproduz numericamente o comportamento de seis algoritmos de escalonamento de tarefas em um processador. A entrada consiste em um conjunto de tarefas, cada uma descrita por identificador, instante de ingresso, tempo de processamento e prioridade; opcionalmente, uma tarefa pode declarar o uso de um recurso de uso exclusivo por meio de uma seção crítica. A saída compreende as métricas individuais de cada tarefa (tempo de execução, tempo de processamento, tempo de espera e tempo até a primeira execução), as respectivas médias, a eficiência do escalonador quando aplicável e o diagrama de tempo no formato linha-por-tarefa.

Os algoritmos implementados são FCFS (First-Come, First-Served), SJF (Shortest Job First), SRTF (Shortest Remaining Time First), Round-Robin, prioridade cooperativa (PRIOc) e prioridade preemptiva (PRIOp). Além desses, o simulador reproduz o fenômeno da inversão de prioridades e oferece dois mecanismos de correção — herança de prioridade e teto de prioridade — bem como envelhecimento (aging) para mitigar a inanição de tarefas de baixa prioridade.

O programa é distribuído como executável independente gerado com PyInstaller (dist/SimuladorEscalonamento.exe), não exigindo instalação de intérprete nem de bibliotecas externas. A interface gráfica é construída com Tkinter; os diagramas de tempo são produzidos com Matplotlib. A arquitetura segue o padrão MVC, com modelo em `model/`, controlador em `control/` e apresentação em `view/`.

SEPARAÇÃO ENTRE POLÍTICA E MECANISMO

A arquitetura do simulador separa o mecanismo de simulação da política de escolha da tarefa. O mecanismo — avanço do relógio, montagem do conjunto de tarefas prontas, cobrança da troca de contexto, suspensão por recurso e cálculo de métricas — é comum a todos os algoritmos. A política — o critério que seleciona qual tarefa recebe o processador em cada instante — é o único ponto que difere entre as seis implementações.

O laço principal de simulação (mecanismo) reside em cada arquivo de algoritmo sob `control/algoritmos/`, mas o critério de seleção de tarefa (política) é o único trecho que muda de um arquivo para outro. O módulo `control/simular_escalonamento.py` atua como roteador: recebe a lista de processos, o identificador numérico do algoritmo e os parâmetros de

configuração (quantum, custo de troca e alpha) e despacha a chamada para a implementação correspondente.

Cada algoritmo retorna as métricas calculadas (médias de tempo de execução, de espera e de tempo até a primeira execução, além do nome do algoritmo) sem conhecer a interface gráfica. A interface, por sua vez, não conhece o critério de escolha: limita-se a coletar a entrada do usuário, invocar o roteador e encaminhar o resultado para a apresentação do diagrama de tempo e da tabela de métricas em `view/graficoProcessos.py`. Essa separação permite acrescentar um novo algoritmo sem alterar a interface nem o modelo de tarefa.

DIAGRAMA DE MÓDULOS

A Figura 1 apresenta o fluxo de dados entre os módulos do simulador, da entrada das tarefas até a apresentação das métricas e do diagrama de tempo. O gerador de cenários e a simulação em lotes alimentam o mesmo caminho de execução, reutilizando o controlador e os algoritmos.

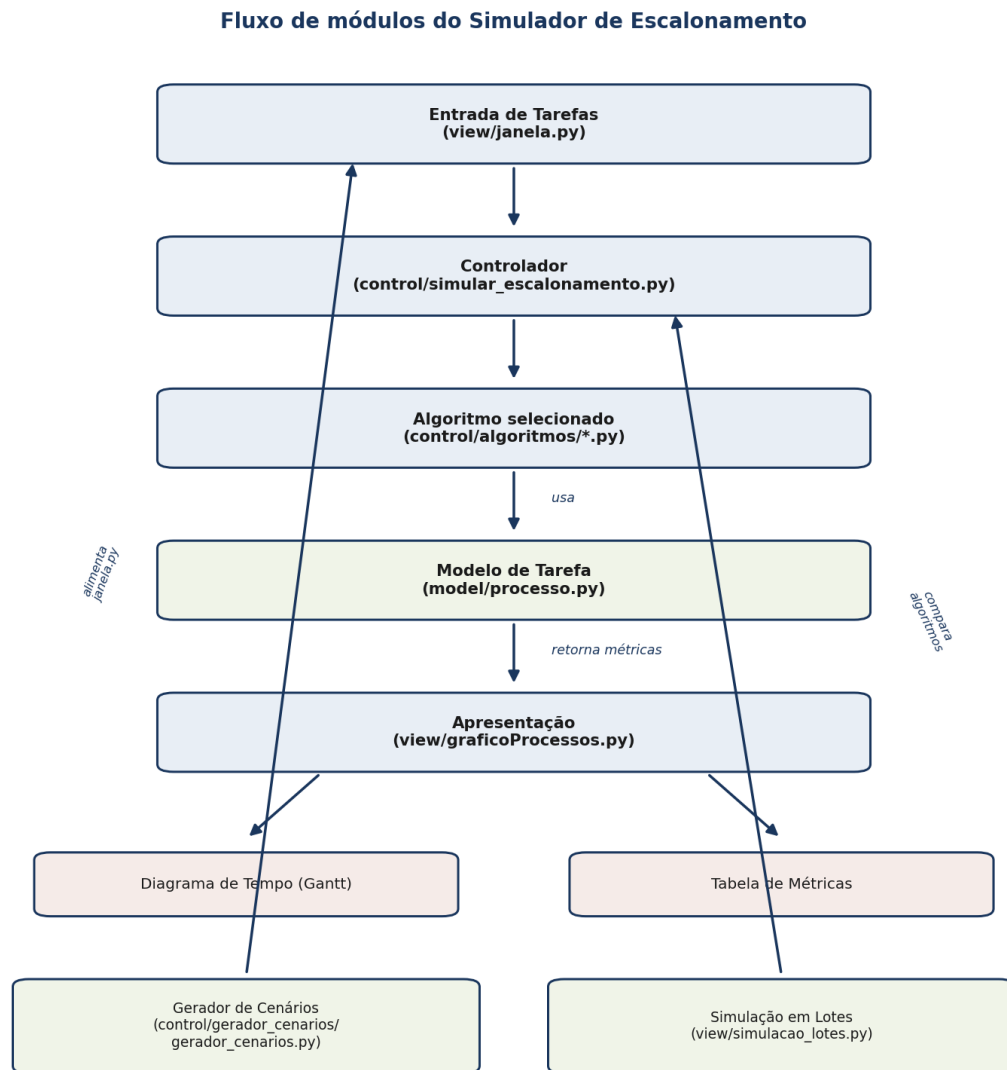


Figura 1 – Diagrama de módulos e fluxo de dados do simulador.

A entrada de tarefas ocorre em `view/janela.py`, que instancia objetos `Processo` e os envia ao controlador. O controlador (`control/simular_escalonamento.py`) seleciona o algoritmo e devolve as médias. O algoritmo selecionado opera sobre o modelo (`model/processo.py` e `model/prioridade.py`) e registra períodos de execução e de troca de contexto. A apresentação (`view/graficoProcessos.py`) consome esses períodos para montar o diagrama de Gantt e a tabela de métricas.

O gerador de cenários (`control/gerador_cenarios/gerador_cenarios.py`) produz listas aleatórias de `Processo` e as injeta na janela principal. A simulação em lotes (`view/simulacao_lotes.py`) percorre um conjunto de cenários sorteados e compara o

desempenho de todos os algoritmos sobre o mesmo lote, reutilizando o roteador de escalonamento.

ESTRUTURA DE UMA TAREFA

A classe `Processo`, definida em `model/processo.py`, concentra o estado de uma tarefa ao longo da simulação. A Tabela 1 descreve cada campo, seu tipo e a função que desempenha no modelo.

Tabela 1 – Campos da classe `Processo` (`model/processo.py`).

Campo	Tipo	Função
<code>id</code>	inteiro	Identificador único da tarefa
<code>chegada</code>	inteiro	Instante em que a tarefa ingressa na fila
<code>duracao</code>	inteiro	Tempo total de processamento demandado (tp)
<code>prioridade</code>	Prioridade	Objeto que encapsula o valor numérico de prioridade; valor maior = prioridade mais alta
<code>sc_inicio</code>	inteiro ou None	Unidade de CPU executada pela tarefa ao obter o recurso R
<code>sc_duracao</code>	inteiro ou None	Número de unidades de CPU durante as quais a tarefa mantém R
<code>sc_fim</code>	inteiro ou None	$sc_inicio + sc_duracao$; calculado no construtor
<code>tempo_restante</code>	inteiro	Tempo de processamento ainda não executado
<code>tempo_executado</code>	inteiro	Tempo de CPU já consumido
<code>prioridade_base</code>	inteiro	Prioridade original, imutável
<code>prioridade_efetiva</code>	inteiro	Prioridade em uso no momento; pode ser elevada por herança ou teto
<code>inicio_espera</code>	inteiro	Instante em que a tarefa começou a aguardar (para aging)
<code>bloqueado</code>	booleano	Indica se a tarefa está suspensa aguardando o recurso R

A classe `Periodo` registra cada intervalo de execução ou de troca de contexto associado a uma tarefa. Seus campos são `inicio` (instante de início do intervalo), `fim` (instante de término) e `tipo`, cujo valor é "Execução" ou "CTX". Os períodos ficam armazenados na lista `processamentos` de cada `Processo` e alimentam tanto o cálculo das métricas quanto a construção do diagrama de tempo.

A classe Prioridade, em model/prioridade.py, encapsula o valor numérico da prioridade (atributo numero) e associa, por convenção interna, um rótulo de recurso ("A" ou "B") com base na paridade do número, mantendo compatibilidade com os módulos de herança e inversão de prioridade.

INTERFACE DOS MÓDULOS

Esta seção descreve as funções e classes principais do simulador: o que cada uma recebe, o que devolve e o efeito que produz sobre o estado da simulação.

Processo.__init__(id, chegada, duracao, prioridade, sc_inicio, sc_duracao)

Inicializa todos os campos da tarefa. Quando sc_inicio e sc_duracao são informados, calcula $sc_fim = sc_inicio + sc_duracao$; caso contrário, os três campos de seção crítica permanecem None. Define prioridade_base e prioridade_efetiva como iguais ao valor numérico inicial extraído do objeto Prioridade. Inicializa tempo_restante com duracao, tempo_executado com zero, processamentos como lista vazia, inicio_espera com chegada e bloqueado como False.

Processo.adicionar_processamento(inicio, fim) → duracao_real

Registra um período de execução do tipo "Execução". Se o intervalo solicitado exceder o tempo restante, trunca fim de modo que a duração efetiva seja igual a tempo_restante. Desconta a duração de tempo_restante, incrementa tempo_executado e devolve a duração real registrada.

Processo.adicionar_troca_contexto(inicio, fim) → duracao

Registra um período do tipo "CTX". A troca de contexto não consome tempo de CPU do processo: não altera tempo_restante nem tempo_executado. Se $fim \leq inicio$, devolve zero e não registra período.

Processo.precisa_recurso() → bool

Retorna True quando $sc_inicio \leq tempo_executado < sc_fim$, indicando que a tarefa se encontra dentro da janela de seção crítica e, portanto, solicita ou detém o recurso R. Retorna False se a seção crítica não foi declarada ou se o tempo executado está fora da janela.

Processo.get_turnaround() → int

Retorna `processamentos[-1].fim - chegada`, isto é, o tempo de execução (turnaround) da tarefa. Se não houver períodos registrados, devolve zero.

Processo.get_espera() → int

Retorna `get_turnaround() - duracao`. Conforme a convenção C8 do enunciado, o tempo de espera engloba fila, suspensão no recurso e troca de contexto.

Processo.get_tempo_primeira_execucao() → int

Filtra os períodos do tipo "Execução" e retorna o instante de início do primeiro deles menos chegada. Se não houver período de execução, devolve zero.

gerar_cenario(quantidade, chegada_max, duracao_max, prioridade_max) → list[Processo]

Função definida em `control/gerador_cenarios/gerador_cenarios.py`. Sorteia quantidade tarefas com chegada $\in [0, chegada_max]$, duracao $\in [1, duracao_max]$ e prioridade $\in [1, prioridade_max]$. Cada tarefa recebe um identificador sequencial a partir de 1. A lista resultante é ordenada por id antes de ser devolvida.

simular_escalonamento(processos, algoritmo, quantum_entry, ctx_time, alpha) → (media_execucao, media_espera, media_primeira_execucao, nome_processo)

Função definida em `control/simular_escalonamento.py`. Recebe a lista de processos e os parâmetros de configuração. Despacha para o algoritmo identificado pelo inteiro `algoritmo`, conforme o mapeamento: 1 = FCFS, 2 = SJF, 3 = Round-Robin, 4 = SRTF, 5 = PRIOf, 6 = PRIOp, 7 = Inversão de prioridade, 8 = Herança de prioridade e 9 = Teto de prioridade. Retorna as médias calculadas e o nome do algoritmo para apresentação. Levanta exceção se a lista de processos estiver vazia ou se o identificador do algoritmo for inválido.

PARÂMETROS CONFIGURÁVEIS

Os parâmetros de tempo e de correção são definidos por quem executa o simulador, pela interface gráfica, e não ficam fixados no código-fonte. A Tabela 2 resume a descrição, a faixa válida e o valor padrão de cada um.

Tabela 2 – Parâmetros configuráveis do simulador.

Parâmetro	Descrição	Faixa válida	Padrão
Número de tarefas	Quantidade de tarefas a simular	≥ 1	—
Quantum (tq)	Duração da fatia de tempo no Round-Robin	$>$ custo de troca	2
Custo de troca (ttc)	Overhead de cada troca de contexto	≥ 0	0
Alpha (α)	Passo de envelhecimento por unidade de tempo de espera	≥ 0	0
Mecanismo de correção	Nenhum, Herança de Prioridade ou Teto de Prioridade	$\{0, 1, 2\}$	Nenhum

Quando o quantum é informado sob Round-Robin e $\text{quantum} \leq \text{custo de troca}$, o programa recusa o valor com mensagem de erro e solicita nova entrada. Essa validação ocorre em `view/janela.py`, antes da chamada ao roteador de escalonamento. O campo alpha só é exibido quando o algoritmo selecionado é PRIOc ou PRIOp. O quantum só é exibido quando o algoritmo selecionado é Round-Robin.

FUNCIONAMENTO INTERNO

Esta seção descreve o comportamento do laço de simulação a cada unidade de tempo e o cálculo das métricas ao término de todas as tarefas.

Laço de simulação

A cada unidade de tempo t , o mecanismo executa a sequência abaixo.

1. As tarefas com chegada $\leq t$, $\text{tempo_restante} > 0$ e não bloqueadas compõem o conjunto de prontas.
2. A política seleciona a tarefa a executar segundo o critério do algoritmo (menor ingresso no FCFS, menor tp no SJF, menor tp restante no SRTF, primeiro da fila circular no Round-Robin, maior prioridade efetiva em PRIOc e PRIOp).
3. Se a tarefa selecionada for diferente da que ocupava o processador no instante anterior, registra-se uma troca de contexto pelo custo ttc . Sob Round-Robin, o custo é descontado da fatia ($tq - ttc$ unidades úteis); nos demais algoritmos, o custo é contabilizado no tempo de execução, mas não subtrai processamento útil.
4. A tarefa executa durante o intervalo determinado pela política: 1 unidade nos algoritmos sem quantum; $tq - ttc$ unidades úteis no Round-Robin quando houve troca.

5. Quando uma tarefa solicita o recurso R (isto é, `precisa_recurso()` retorna True) e outra tarefa o detém, a tarefa solicitante é marcada como bloqueado = True e removida do conjunto de prontas. Ela só retorna ao conjunto quando a detentora libera R.
6. Quando a herança está habilitada: ao bloquear, verifica-se se a detentora possui prioridade inferior à da tarefa bloqueada; se sim, `prioridade_efetiva` da detentora é elevada ao máximo entre as bloqueadas. Ao liberar R, `prioridade_efetiva` retorna a `prioridade_base`.
7. Quando o teto está habilitado: ao obter o recurso, a tarefa assume imediatamente a prioridade teto do recurso (maior prioridade entre todas as tarefas que declaram seção crítica naquele recurso), independentemente de existir conflito.
8. Envelhecimento: a cada unidade de tempo que uma tarefa pronta aguarda sem ser despachada, sua `prioridade_efetiva` cresce α unidades. Ao receber o processador, `prioridade_efetiva` retorna a `prioridade_base`.
9. Se nenhuma tarefa ingressou ainda, o relógio salta para o próximo instante de ingresso, evitando laços ociosos desnecessários.

Métricas

Após o término de todas as tarefas, calculam-se as grandezas individuais e as médias aritméticas.

- $T_t = \text{processamentos}[-1].\text{fim} - \text{chegada}$ (tempo de execução / turnaround)
- $T_p = \text{duracao}$ (tempo de processamento — dado de entrada)
- $T_w = T_t - T_p$ (tempo de espera, conforme C8)
- $T_{1a} = \text{periodos_execucao}[0].\text{inicio} - \text{chegada}$ (tempo até a primeira execução)
- Médias aritméticas de T_t , T_w e T_{1a} sobre todas as tarefas
- Eficiência $E = t_q / (t_q + t_{tc})$ — apresentada apenas para Round-Robin; indicada como "não definida" para os demais algoritmos

As médias são devolvidas pelo roteador à interface, que as exibe juntamente com a tabela por tarefa e o diagrama de tempo. O diagrama representa cada tarefa em uma linha horizontal, com segmentos coloridos para execução e para troca de contexto, permitindo a inspeção visual da ordem de despacho e dos intervalos de espera.

CONVENÇÕES DE SIMULAÇÃO

As dez convenções abaixo são as mesmas adotadas em sala de aula e determinam os valores numéricos produzidos pelo simulador. Divergir de qualquer uma delas produz resultados diferentes dos cenários de referência do enunciado.

C1. O tempo é discreto e avança de uma em uma unidade. Nos cenários das aulas a unidade é o segundo.

C2. Prioridade: valor maior significa prioridade mais alta.

C3. Desempate: menor instante de ingresso; em caso de empate, menor identificador.

C4. Ocorre troca de contexto sempre que a tarefa despachada difere da última que ocupou o processador, inclusive no primeiro despacho.

C5. O custo da troca é descontado da fatia concedida, nunca somado a ela. Sob Round-Robin, a tarefa executa $tq - ttc$ unidades úteis quando houve troca.

C6. A tarefa que esgota o quantum retorna à cauda da fila depois das que ingressaram no mesmo instante.

C7. A seção crítica é medida no tempo de execução própria da tarefa. Início 1 e duração 4 significa que a tarefa obtém o recurso após executar 1 unidade útil e o mantém até ter executado 5.

C8. $T_w = T_t - T_p$. O tempo de espera engloba fila, suspensão no recurso e troca de contexto.

C9. O teto de um recurso é calculado sobre as tarefas que declaram seção crítica nele, ainda que a disputa não ocorra. Herança e teto são protocolos alternativos e não se combinam.

C10. O envelhecimento conta o tempo decorrido desde o último despacho da tarefa, ou desde o ingresso caso ela ainda não tenha executado.

DEPENDÊNCIAS E AMBIENTE

O simulador pode ser utilizado de duas formas: por meio do executável independente ou a partir do código-fonte.

Executável

O arquivo `dist/SimuladorEscalonamento.exe` não requer instalação de Python nem de bibliotecas externas. É gerado com PyInstaller a partir do arquivo de especificação `SimuladorEscalonamento.spec`. É compatível com Windows 10 ou superior (64 bits). A abertura se dá por duplo clique sobre o executável.

Execução via código-fonte

A execução a partir do código-fonte exige Python 3.10 ou superior. O Tkinter integra a instalação padrão do Python e fornece a interface gráfica. O Matplotlib é utilizado para a geração dos diagramas de tempo. As demais dependências constam em requirements.txt, entre as quais numpy, pillow, pandas e seaborn.

A instalação das dependências é realizada com o comando:

```
pip install -r requirements.txt
```

O ponto de entrada do código-fonte é main.py, que invoca a função de criação da janela principal em view/janela.py. A estrutura de pastas do repositório separa modelo (model/), controlador (control/ e control/algoritmos/), apresentação (view/), recursos visuais (assets/), cenários exportados (cenarios/) e documentação (docs/).

REFERÊNCIAS

TANENBAUM, Andrew S.; BOS, Herbert. **Sistemas operacionais modernos**. 4. ed. São Paulo: Pearson, 2016.

SILBERSCHATZ, Abraham; GALVIN, Peter B.; GAGNE, Greg. **Fundamentos de sistemas operacionais**. 9. ed. Rio de Janeiro: LTC, 2015.

PYTHON SOFTWARE FOUNDATION. **The Python Standard Library**. Disponível em: <https://docs.python.org/3/library/>. Acesso em: set. 2026.

PYINSTALLER DEVELOPMENT TEAM. **PyInstaller Manual**. Disponível em: <https://pyinstaller.org/en/stable/>. Acesso em: set. 2026.